

Software Engineering I

Assignment 3

Fabian Gonzalez

September 26, 2026

1. The two GenAI programs that I used for this assignment are ClaudeAI and Gemini.
2. I accessed both of them through the google search window. Both Gemini and Claude can be accessed by searching them up and locating them on the Google search engine.
3. Prompt used for both GenAI models:

Create a program in C++ using objects, not functions, that will prioritize emails for a busy company CEO. MaxHeap must be used as a means of implementing a priority queue, where emails can shift towards the front of the queue based on priority status. You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your prioritizing for the CEO. Create objects EMAIL, NEXT, READ, and count from scratch:

EMAIL <sender category>, <subject line>, <date> - The emails in the CEO's Inbox should be placed in queue based on their sender category and date. Sender categories and priorities should be as followed:

Boss - read first

Subordinate - read next

Peer - read next

ImportantPerson - read next

OtherPerson - read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first.

EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas.

Assume <date> is in the format: MM-DD-YYYY

NEXT - Next email for the CEO to read. Display the information in the terminal in the following format:

Sender:

Subject:

Date:

READ - CEO has read email and dealt with it

COUNT - Display current count of unread emails

EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas.

Assume <date> is in the format: MM-DD-YYYY

The code must be tested in a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

Example Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT
NEXT
READ
NEXT
READ
COUNT

Example Output:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

4. Generated code from both GenAIs:

Gemini

:

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <algorithm>
```

```
// Class to represent a single Email
```

```
class Email {
```

```
Private:
```

```
    std::string senderCategory;
```

```
    std::string subject;
```

```
    std::string dateStr; // Format: MM-DD-YYYY
```

```
    int categoryPriority;
```

```
    long long formattedDate; // Format: YYYYMMDD for easy date comparison
```

```
// Maps sender category strings to integer priorities (Higher = higher priority)
```

```
int parseCategoryPriority(const std::string& category) const {
```

```
    if (category == "Boss") return 5;
```

```
    if (category == "Subordinate") return 4;
```

```
    if (category == "Peer") return 3;
```

```
    if (category == "ImportantPerson") return 2;
```

```
    if (category == "OtherPerson") return 1;
```

```
    return 0;
```

```
}
```

```

// Converts MM-DD-YYYY string into a numeric YYYYMMDD representation
long long parseDateToNumeric(const std::string& dStr) const {
    if (dStr.length() < 10) return 0;

    std::string month = dStr.substr(0, 2);
    std::string day = dStr.substr(3, 2);
    std::string year = dStr.substr(6, 4);

    return std::stoll(year + month + day);
}

public:
    Email() : categoryPriority(0), formattedDate(0) {}

    Email(std::string sender, std::string subj, std::string dt)
        : senderCategory(sender), subject(subj), dateStr(dt) {
        categoryPriority = parseCategoryPriority(senderCategory);
        formattedDate = parseDateToNumeric(dateStr);
    }

    std::string getSenderCategory() const { return senderCategory; }
    std::string getSubject() const { return subject; }
    std::string getDate() const { return dateStr; }

// Overloaded operators for heap priority comparisons
// 1. Higher category priority comes first.
// 2. If categories match, newer date comes first.
    bool operator<(const Email& other) const {
        if (this->categoryPriority != other.categoryPriority) {
            return this->categoryPriority < other.categoryPriority;

```

```

    }

    return this->formattedDate < other.formattedDate;
}

bool operator>(const Email& other) const {
    if (this->categoryPriority != other.categoryPriority) {
        return this->categoryPriority > other.categoryPriority;
    }

    return this->formattedDate > other.formattedDate;
}
};

// Class implementing a MaxHeap using a dynamic list container
class MaxHeap {
private:
    std::vector<Email> heapList;

    void heapifyUp(int index) {
        while (index > 0) {
            int parentIndex = (index - 1) / 2;
            if (heapList[index] > heapList[parentIndex]) {
                std::swap(heapList[index], heapList[parentIndex]);
                index = parentIndex;
            } else {
                break;
            }
        }
    }
}

```

```

void heapifyDown(int index) {
    int size = heapList.size();
    while (index < size) {
        int leftChild = 2 * index + 1;
        int rightChild = 2 * index + 2;
        int largest = index;

        if (leftChild < size && heapList[leftChild] > heapList[largest]) {
            largest = leftChild;
        }
        if (rightChild < size && heapList[rightChild] > heapList[largest]) {
            largest = rightChild;
        }

        if (largest != index) {
            std::swap(heapList[index], heapList[largest]);
            index = largest;
        } else {
            break;
        }
    }
}

```

public:

```

    MaxHeap() {}

```

```

    void insert(const Email& email) {

```

```

        heapList.push_back(email);

        heapifyUp(heapList.size() - 1);
    }

    Email getMax() const {
        if (isEmpty()) {
            throw std::out_of_range("Heap is empty.");
        }
        return heapList[0];
    }

    void removeMax() {
        if (isEmpty()) return;
        heapList[0] = heapList.back();
        heapList.pop_back();
        if (!isEmpty()) {
            heapifyDown(0);
        }
    }

    bool isEmpty() const {
        return heapList.empty();
    }

    int size() const {
        return heapList.size();
    }
};

```

```
// CEO Email Manager Object encapsulating command logic
```

```
class CEOEmailManager {
```

```
private:
```

```
    MaxHeap emailQueue;
```

```
public:
```

```
    CEOEmailManager() {}
```

```
    void processEmailCommand(const std::string& args) {
```

```
        std::stringstream ss(args);
```

```
        std::string sender, subject, date;
```

```
        if (std::getline(ss, sender, ',') &&
```

```
            std::getline(ss, subject, ',') &&
```

```
            std::getline(ss, date)) {
```

```
            Email newEmail(sender, subject, date);
```

```
            emailQueue.insert(newEmail);
```

```
        }
```

```
    }
```

```
    void processNextCommand() const {
```

```
        if (emailQueue.isEmpty()) {
```

```
            std::cout << "No emails to display.\n\n";
```

```
            return;
```

```
        }
```

```
        Email top = emailQueue.getMax();
```

```
        std::cout << "Next email:\n";
```

```

        std::cout << "Sender: " << top.getSenderCategory() << "\n";

        std::cout << "Subject: " << top.getSubject() << "\n";

        std::cout << "Date: " << top.getDate() << "\n\n";
    }

void processReadCommand() {
    if (!emailQueue.isEmpty()) {
        emailQueue.removeMax();
    }
}

void processCountCommand() const {
    std::cout << "There are " << emailQueue.size() << " emails to read.\n\n";
}

void parseAndExecute(const std::string& commandLine) {
    if (commandLine.empty()) return;

    if (commandLine.rfind("EMAIL ", 0) == 0) {
        processEmailCommand(commandLine.substr(6));
    } else if (commandLine == "NEXT") {
        processNextCommand();
    } else if (commandLine == "READ") {
        processReadCommand();
    } else if (commandLine == "COUNT") {
        processCountCommand();
    }
}

```

```
};
```

```
int main() {  
    // Simulated input commands from the test file example  
    std::vector<std::string> testCommands = {  
        "EMAIL Peer,Can you help me on this?,12-01-2024",  
        "EMAIL OtherPerson,Try our product,12-19-2024",  
        "EMAIL Boss,Important,12-20-2024",  
        "EMAIL Subordinate,How do I handle this?,12-25-2024",  
        "EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024",  
        "EMAIL Boss,Never Mind,01-03-2025",  
        "COUNT",  
        "NEXT",  
        "READ",  
        "NEXT",  
        "READ",  
        "COUNT"  
    };  
  
    CEOEmailManager manager;  
  
    for (const auto& command : testCommands) {  
        manager.parseAndExecute(command);  
    }  
    return 0;  
}
```

Claude:

```

// CEO Inbox Prioritizer
// Usage: ./ceo_inbox <test_file>    (or pipe a file via stdin)
//
// Design: everything is an object.
// List<T> - from-scratch growable list (array-backed) used as heap storage
// MaxHeap<T>- max-heap priority queue built on List<T>
// Email    - sender category, subject, date, plus its priority comparison
// Inbox    - owns the MaxHeap<Email>; the CEO's unread mail
// Command  - abstract base; NextCommand, ReadCommand, CountCommand are
//            the NEXT / READ / COUNT objects. EmailCommand adds an EMAIL.
// CommandParser - turns a text line into a Command object
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <memory>
#include <stdexcept>

// ----- List -----
template <typename T>
class List {
    T*    data_;
    size_t size_;
    size_t cap_;
    void grow() {
        size_t newCap = cap_ * 2;
        T* bigger = new T[newCap];
        for (size_t i = 0; i < size_; ++i) bigger[i] = data_[i];
        delete[] data_;
        data_ = bigger;
        cap_ = newCap;
    }
public:
    List() : data_(new T[8]), size_(0), cap_(8) {}
    List(const List& o) : data_(new T[o.cap_]), size_(o.size_), cap_(o.cap_) {
        for (size_t i = 0; i < size_; ++i) data_[i] = o.data_[i];
    }

```

```

List& operator=(const List& o) {
    if (this != &o) {
        delete[] data_;
        data_ = new T[o.cap_];
        size_ = o.size_;
        cap_ = o.cap_;
        for (size_t i = 0; i < size_; ++i) data_[i] = o.data_[i];
    }
    return *this;
}

~List() { delete[] data_; }

void pushBack(const T& v) { if (size_ == cap_) grow(); data_[size_++] = v; }
void popBack()          { if (size_ > 0) --size_; }
T& operator[](size_t i)  { return data_[i]; }
const T& operator[](size_t i) const { return data_[i]; }
T& back()               { return data_[size_ - 1]; }
size_t size() const     { return size_; }
bool empty() const      { return size_ == 0; }
};

```

```

// ----- MaxHeap -----
// T must provide: bool higherPriorityThan(const T&) const
template <typename T>
class MaxHeap {
    List<T> items_;

    static size_t parent(size_t i) { return (i - 1) / 2; }
    static size_t left(size_t i)  { return 2 * i + 1; }
    static size_t right(size_t i) { return 2 * i + 2; }

    void swapAt(size_t a, size_t b) {
        T tmp = items_[a]; items_[a] = items_[b]; items_[b] = tmp;
    }

    void siftUp(size_t i) {
        while (i > 0 && items_[i].higherPriorityThan(items_[parent(i)])) {
            swapAt(i, parent(i));

```

```

        i = parent(i);
    }
}

void siftDown(size_t i) {
    size_t n = items_.size();
    while (true) {
        size_t best = i, l = left(i), r = right(i);
        if (l < n && items_[l].higherPriorityThan(items_[best])) best = l;
        if (r < n && items_[r].higherPriorityThan(items_[best])) best = r;
        if (best == i) break;
        swapAt(i, best);
        i = best;
    }
}

public:
    void insert(const T& v) { items_.pushBack(v); siftUp(items_.size() - 1); }
    const T& peekMax() const {
        if (items_.empty()) throw std::out_of_range("heap empty");
        return items_[0];
    }
    void removeMax() {
        if (items_.empty()) throw std::out_of_range("heap empty");
        items_[0] = items_.back();
        items_.popBack();
        if (!items_.empty()) siftDown(0);
    }
    size_t size() const { return items_.size(); }
    bool empty() const { return items_.empty(); }
};

```

// ----- Email -----

```

class Email {
    std::string category_;
    std::string subject_;
    std::string date_;    // MM-DD-YYYY, kept as given for display
    int rank_;           // higher = more important
    int dateKey_;        // YYYYMMDD, higher = newer

```

```

long    seq_;    // arrival order, breaks exact ties (earlier first)

static int rankOf(const std::string& c) {
    if (c == "Boss")        return 5;
    if (c == "Subordinate")  return 4;
    if (c == "Peer")         return 3;
    if (c == "ImportantPerson") return 2;
    return 1;                // OtherPerson
}

static int dateKeyOf(const std::string& d) { // MM-DD-YYYY -> YYYYMMDD
    int mm = std::stoi(d.substr(0, 2));
    int dd = std::stoi(d.substr(3, 2));
    int yy = std::stoi(d.substr(6, 4));
    return yy * 10000 + mm * 100 + dd;
}

public:
    Email() : rank_(0), dateKey_(0), seq_(0) {}
    Email(const std::string& category, const std::string& subject,
          const std::string& date, long seq)
        : category_(category), subject_(subject), date_(date),
          rank_(rankOf(category)), dateKey_(dateKeyOf(date)), seq_(seq) {}

    bool higherPriorityThan(const Email& o) const {
        if (rank_ != o.rank_) return rank_ > o.rank_; // category first
        if (dateKey_ != o.dateKey_) return dateKey_ > o.dateKey_; // newest first
        return seq_ < o.seq_; // stable tie-break
    }

    void display(std::ostream& out) const {
        out << "Sender: " << category_ << "\n"
            << "Subject: " << subject_ << "\n"
            << "Date: " << date_ << "\n";
    }
};

// ----- Inbox -----

class Inbox {
    MaxHeap<Email> unread_;

```

```

    long        nextSeq_;
public:
    Inbox() : nextSeq_(0) {}
    void receive(const std::string& cat, const std::string& subj, const std::string& date) {
        unread_.insert(Email(cat, subj, date, nextSeq_++));
    }
    bool  hasMail() const { return !unread_.empty(); }
    size_t count()  const { return unread_.size(); }
    const Email& peek() const { return unread_.peekMax(); }
    void  removeTop()    { unread_.removeMax(); }
};

// ----- Commands -----
class Command {
public:
    virtual ~Command() {}
    virtual void execute(Inbox& inbox, std::ostream& out) = 0;
};

class EmailCommand : public Command {
    std::string cat_, subj_, date_;
public:
    EmailCommand(const std::string& c, const std::string& s, const std::string& d)
        : cat_(c), subj_(s), date_(d) {}
    void execute(Inbox& inbox, std::ostream&) override { inbox.receive(cat_, subj_, date_); }
};

class NextCommand : public Command { // NEXT: show top email, do not remove
public:
    void execute(Inbox& inbox, std::ostream& out) override {
        if (!inbox.hasMail()) { out << "No emails to read.\n"; return; }
        out << "Next email:\n";
        inbox.peek().display(out);
    }
};

class ReadCommand : public Command { // READ: top email handled, remove it

```

```

public:
    void execute(Inbox& inbox, std::ostream&) override {
        if (inbox.hasMail()) inbox.removeTop();
    }
};

class CountCommand : public Command { // COUNT: unread total
public:
    void execute(Inbox& inbox, std::ostream& out) override {
        out << "There are " << inbox.count() << " emails to read.\n";
    }
};

// ----- Parser ----
class CommandParser {
    static std::string trim(const std::string& s) {
        size_t a = s.find_first_not_of(" \t\r\n");
        if (a == std::string::npos) return "";
        size_t b = s.find_last_not_of(" \t\r\n");
        return s.substr(a, b - a + 1);
    }
public:
    // Returns nullptr for blank/unrecognized lines.
    std::unique_ptr<Command> parse(const std::string& rawLine) const {
        std::string line = trim(rawLine);
        if (line == "NEXT") return std::unique_ptr<Command>(new NextCommand());
        if (line == "READ") return std::unique_ptr<Command>(new ReadCommand());
        if (line == "COUNT") return std::unique_ptr<Command>(new CountCommand());
        if (line.compare(0, 6, "EMAIL ") == 0) {
            std::string rest = line.substr(6);
            size_t c1 = rest.find(',');
            size_t c2 = rest.rfind(',');
            if (c1 == std::string::npos || c2 == c1) return nullptr;
            std::string cat = trim(rest.substr(0, c1));
            std::string subj = trim(rest.substr(c1 + 1, c2 - c1 - 1));
            std::string date = trim(rest.substr(c2 + 1));
            if (date.size() != 10) return nullptr;

```

```

        return std::unique_ptr<Command>(new EmailCommand(cat, subj, date));
    }
    return nullptr;
}
};

// ----- App -----
class App {
    Inbox    inbox_;
    CommandParser parser_;
public:
    void run(std::istream& in, std::ostream& out) {
        std::string line;
        while (std::getline(in, line)) {
            std::unique_ptr<Command> cmd = parser_.parse(line);
            if (cmd) cmd->execute(inbox_, out);
        }
    }
};

int main(int argc, char* argv[]) {
    App app;
    if (argc > 1) {
        std::ifstream file(argv[1]);
        if (!file) { std::cerr << "Cannot open " << argv[1] << "\n"; return 1; }
        app.run(file, std::cout);
    } else {
        app.run(std::cin, std::cout);
    }
    return 0;
}

```

5. Comparing the two in terms of correctness, execution time, space complexity, and maintainability

Correctness:

Claude - Program contains a MaxHeap method to implement a priority queue using list-based implementation. MaxHeap logic in the program works as intended whether the program does or does not

have one element. Priority ordering is handled correctly. The program reads in order of boss, subordinate, peer, important person, other person. Date handling is handled correctly and reads the newest email first. NEXT command does not remove previous email, READ command does remove previous email.

Errors discovered in the program are how the date is entered in the program. If the date is not entered in a standard date format, the program will immediately crash. Other minor things to look out for in the code are unhandled or unexpected exceptions within the code. This will cause the program to stop abruptly.

Gemini - Program contains a MaxHeap method to implement a priority queue using list-based implementation. MaxHeap logic in the program works as intended whether the program does or does not have one element. Priority ordering is handled correctly. The program reads in order of boss, subordinate, peer, important person, other person. Date handling is handled correctly and reads the newest email first. NEXT command does not remove previous email, READ command does remove previous email.

Errors discovered in the program are how the program is unable to read a test file and mainly focuses if the test file is already implemented in the code. Because of this, we would have to change the code in order for the program to read different test files and not just the example. Additionally, the program does not create objects for NEXT, READ, and COUNT but naturally creates them as functions, which goes against the rules of the assignment.

Time Execution:

- Claude - Best-case scenario: $O(n)$. Worst-case scenario: $O(n \log n)$
- Gemini - Best-case scenario: $O(n)$. Worst-case scenario: $O(n \log n)$

When comparing their execution times, both programs have the same best-case and worst-case scenario execution times in Big-O notation. This is because they mainly rely on the test input they put in. If the programming runs as intended, then their Big-O notations would be n . Otherwise, the program would have to process for longer based on if the user inserts or deletes anything.

Space Complexity:

- Claude - Best-case scenario: $O(n)$. Worst-case scenario: $O(2n)$
- Gemini - Best-case scenario: $O(n)$. Worst-case scenario: $O(n)$

Both programs have the same space complexity at their best-case scenarios. In terms of their worst case scenarios, however, Gemini is twice as fast as Claude, making it faster and in turn more useful.

Maintainability:

Claude - Program has good separation of Email, MaxHeap, and CEOEmailManager. The program also contains helpful comments detailing what certain sections of the code does. Its major flaws, however, are its date passing statement which allows it to crash if formatted differently from how its intended, and there is a general lack of security in the program if an exception is passed onto the heap.

Gemini - Program has good separation of Email, MaxHeap, and CEOEmailManager. The program also contains helpful comments detailing what certain sections of the code does. Its major flaws, however, are that the commands are dispatched by a string if/else chain that can be fixed by adding a command class. There is also a lack of error handling which makes it more prone to crashing and runtime error.

6. The program that I will use for the basis of my assignment will be the Claude code. Despite it having a slower worst-case scenario time than the Gemini code, it was able to create separate objects for EMAIL, NEXT, READ, and COUNT. Additionally it does not contain as many bugs and errors as the Gemini code, which makes it more competent for improvement and maintainability.
7. I improved the correctness of the code by making sure that the dates, no matter the format entered, would not crash the system as they are now parsed by hand and checked for format, months, and realdays.

I improved the execution time of the code by using MaxHeap to shift elements out from the heap and onto another with one swap instead of having to use multiple copies to swap.

I improved the space complexity of the code by releasing a removed email's string memory once its fully erased from the list.

I improved the maintainability of the code by removing the if/else chain in the code and replacing it with a command, and avoiding any unnecessary copies from appearing if they are ever made.